



SmartCity

How It Works — A Simple Guide

Every feature explained in plain English, step by step.

Who this guide is for

Anyone who can write a little code. You do *not* need to know FastAPI, React, or databases. Every technical word is explained the first time it appears, and every feature is told as a story: what the user sees, then what the computer does behind the scenes.

Seminar companion · Team HackInMotion · RICR-HIM-1041

Part 1 — What are we even building?

The problem in the real world, before any code

1.1 A story you already know

There is a pothole on your street. It has been there for two months.

You call the municipal office. Someone writes it in a register. You never hear anything again. Your neighbour calls too. And the person across the road. And twelve other people. Each call becomes its own entry in the register.

Now put yourself on the other side of that desk. You are the officer. Your list has fifteen entries that all say “pothole”. Are they the same pothole or fifteen different ones? You cannot tell. You do not know which is most urgent. And nobody outside the office can see whether you are doing a good job or not.

SmartCity in one sentence. It is a website where citizens report civic problems on a map, and the city fixes them — where the computer does the boring, error-prone parts automatically: spotting repeats, choosing the right department, deciding what is urgent, and chasing anything that is late.

1.2 The four problems we solve

Problem	Our answer
The same problem reported many times	The computer checks whether a new report is close to an existing one, of the same type, and recent. If yes, it joins that report instead of making a new one.
Citizens hear nothing back	Every report gets a tracking number and a live status. The citizen sees every step, and gets the final say on whether it was really fixed.
Humans doing routing and triage by hand	The type of problem decides the department automatically. A score decides the order of work automatically.
No one can measure the city	A public web page grades every department from A+ to D, calculated from their own record. No login needed to see it.

1.3 Three kinds of people use it

 **The citizen**
Reports problems from their phone, in English or Hindi. Sees a map of everything reported nearby. Tracks their own reports.

 **The officer**
Sees a work list sorted by urgency, updates status, uploads a photo proving the fix, and watches city-wide charts.

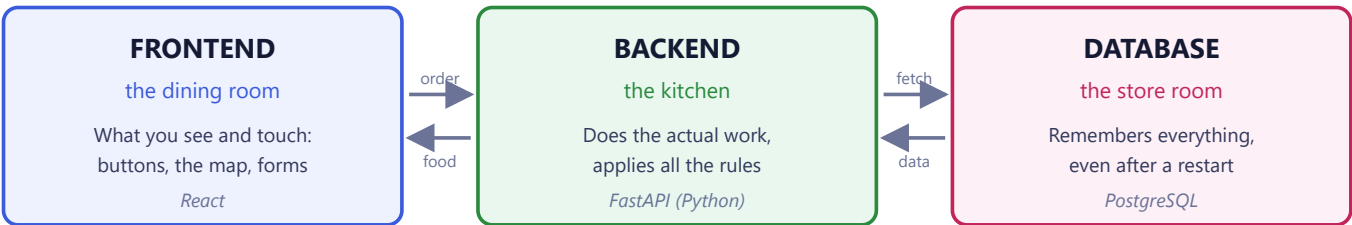
 **The public**
Anyone at all — press, researcher, curious citizen — can open the report card page and see how each department is performing.

Part 2 — The building blocks

What the pieces are, in everyday words

2.1 The restaurant analogy

Almost every website is built from three pieces. A restaurant explains them perfectly.



The customer never enters the kitchen. The kitchen never talks to the customer directly.

The rules are the same in software. The frontend never touches the database. It asks the backend, politely, and the backend decides what is allowed.

Why this separation matters. If the browser could talk to the database directly, anyone could open the developer console and change their own role to “admin”. Because every request must pass through the backend, the backend can check “are you actually allowed to do this?” every single time.

2.2 What is an API?

API — a menu of things the backend can do, each with a fixed address. Instead of “bring me food”, you say `GET /api/citizen/issues`, which means “give me the list of my reports”. The frontend orders from this menu; the backend cooks.

Our menu has about seventeen items grouped into four folders:

Folder	What it holds
<code>/api/auth</code>	Sign up, log in, log out, “who am I?”
<code>/api/citizen</code>	Submit a report, see my reports, the map, vote, confirm or reopen
<code>/api/admin</code>	The work list, charts, updating status, uploading proof
<code>/api/public</code>	The report card — the only one that needs no login

2.3 The tools we used, and why

Tool	What it is	Why we picked it
React	A JavaScript library for building screens out of reusable pieces called components	Write the map once, use it on both the citizen and officer pages
FastAPI	A Python framework for building the backend	Very readable, and it checks incoming data automatically so bad input never reaches our code
PostgreSQL	A database — organised tables of rows, like very strict spreadsheets	Free, reliable, and good at answering “find everything near this point”
Leaflet + OpenStreetMap	A map library plus free map images	No API key, no bill. Google Maps would need a credit card on file
Alembic	Keeps a numbered list of database changes	Any teammate can run one command and get the exact same tables

2.4 What happens when you click a button

- 1 You tap **Submit**. React gathers what you typed into a small package of data.
- 2 The browser sends that package over the internet to a web address on our server.
- 3 FastAPI receives it and first asks: *who is this person, and are they allowed?*
- 4 It checks the data is sensible — is the description long enough? Is the photo really a photo?
- 5 It runs our rules (is this a repeat? which department?) and saves rows into the database.
- 6 It sends back a short answer — usually the new tracking number.
- 7 React reads that answer and updates the screen. Total time: well under a second.

Part 3 — Getting in: accounts and safety

Feature 1 of 16

3.1 What the user sees

A sign-up form asking for email and password. After that, a login form. Citizens sign up themselves. Officer accounts are created by an administrator — you cannot become an officer by ticking a box.

3.2 How the password is stored

We never store your actual password. Ever. We store a scrambled version.

Analogy. Think of blending a fruit into juice. Easy to go from fruit to juice. Impossible to turn the juice back into the fruit. When you log in, we blend what you typed and compare the two juices. If they match, it was the right password — and we still never learned what it was.

bcrypt — the specific blending recipe we use. It is deliberately *slow*, which sounds bad but is the whole point: an attacker who steals the database can only test a few guesses per second instead of millions.

3.3 Staying logged in — what a token is

The website cannot ask for your password on every click. So when you log in successfully, the server gives your browser a **token**.

Analogy. A token is a wristband at an event. The guard checks your ID once at the gate and gives you a wristband. After that you just show the wristband. Ours has an expiry printed on it — after 30 minutes it stops working and you log in again.

The wristband is signed with a secret only our server knows, so nobody can forge one. It also carries your role — citizen or officer — so the server knows what you may do.

3.4 The security rule that actually matters

In our React code, a citizen simply never sees a link to the officer pages. That is **not** security. That is just tidiness. Anyone can type a URL.

Real security lives on the server. Every officer-only address has a guard attached in Python. If a citizen's token arrives at an officer address, the server refuses with an error before running a single line of our logic. Hiding a button is decoration; the server check is the lock.

```
# Every admin route carries this guard. No guard, no access.
def admin_only(user = Depends(current_user)):
    if user.role != Role.ADMIN:
        raise HTTPException(403, "Not allowed")
    return user
```

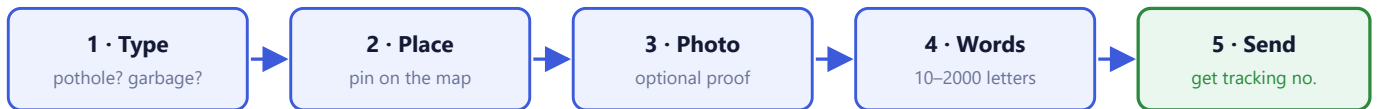
One more small but clever detail: if you ask for a report that belongs to someone else, we reply **“not found”** rather than **“not allowed”**. Saying “not allowed” would quietly confirm that the report exists. “Not found” tells a snooper nothing.

Part 4 — Reporting a problem

Features 2, 3 and 4 — the form, the map pin, the photo

4.1 Five small steps instead of one big form

A long form on a phone is exhausting, and if you get one field wrong at the end you often lose everything. So we split it into five screens, each asking one thing.



Each step is checked before you may continue, so mistakes are caught immediately — next to the thing you got wrong, not in a red list at the bottom.

4.2 Putting a pin on the map

Two ways, because one of them will always fail for somebody:

- **“Use my location”** — the browser asks your phone for GPS coordinates.
- **Tap the map** — you place the pin yourself, and can drag it to adjust.

People deny location permission. GPS fails indoors. So if the first way fails we show a calm message and the map stays fully usable. The report is never blocked.

Turning coordinates into an address

A pin gives us numbers like `23.2599, 77.4126`. Useful to a computer, useless on a work order. So we ask a free service called **Nominatim**: “what is at this point?” It replies with something like *“Main Road, Bhopal”*.

What if that service is down? Then we simply save the report with only the numbers, and show “address not found — coordinates will be used”. A crew can still navigate to a coordinate. **An outside service must never be able to break our form.** You will see this idea repeat all through the project.

4.3 Accepting a photo — safely

Letting strangers upload files to your server is one of the most dangerous things a website can do. We apply four checks in order.

#	Check	Why
1	Is there actually a file?	Catches empty uploads
2	Is it under 5 MB?	Stops someone filling our disk with one giant file
3	Does it claim to be a JPEG, PNG or WEBP?	A quick first filter
4	Do the actual bytes look like an image?	The one that really protects us

Analogy for check 4. Checks 2 and 3 read the label on the parcel. Check 4 opens the box. A file named `photo.jpg` that claims to be an image can still be a program inside. Renaming a file changes nothing about its contents.

Real image formats begin with a fixed signature — a few specific bytes at the very start. We read those bytes ourselves:

Format	First bytes (its “fingerprint”)
JPEG	<code>FF D8 FF</code>
PNG	<code>89 50 4E 47 0D 0A 1A 0A</code>
WEBP	<code>RIFF</code> ... then <code>WEBP</code>

If the fingerprint is wrong, we reject the file no matter what it is called.

And we throw the filename away

We never save a file under the name the user gave it. We generate a fresh random name. This blocks a classic attack where a filename like `../../config.py` tricks the server into overwriting its own files.

Part 5 — The clever part: spotting repeats

Feature 5 — the heart of the whole project ★

This is the feature to spend the most time on in your seminar. It is the piece that turns a complaint box into a real system.

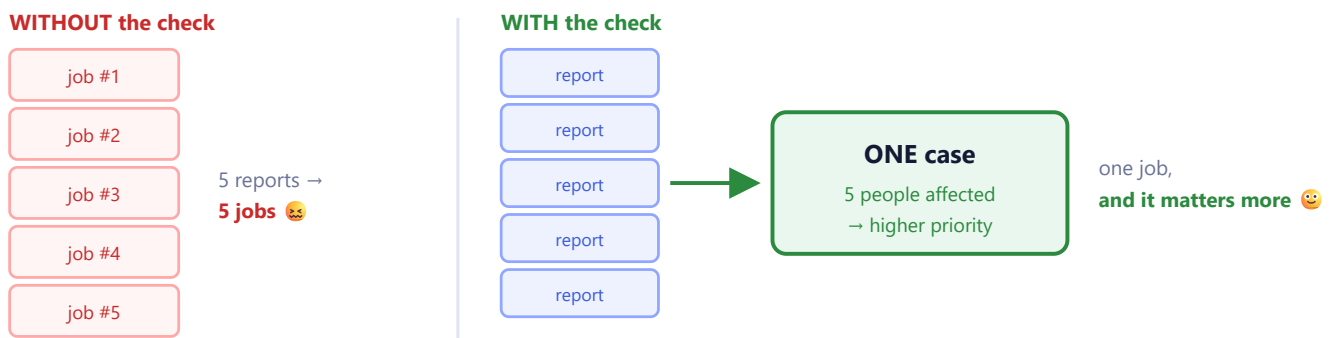
5.1 The idea

When a new report arrives we ask one question before saving it:

“Is somebody already reporting this exact problem?”

If yes, we attach the new report to the existing one instead of creating a second job.

We call the shared job a **case**. Fifteen reports of one pothole become one case with fifteen reporters. The officer sees one row of work. Every reporter still keeps their own tracking number and can follow progress.



The repeats do not disappear — they become evidence that the problem is worse.

5.2 What counts as “the same problem”?

Three things must all be true:

- 1 **Same type.** A pothole is never merged with a broken streetlight.
- 2 **Close enough.** Within a distance that depends on the type (explained below).
- 3 **Recent enough.** Within a time window that also depends on the type.

Plus one more: the existing case must still be **open**. If a pothole was fixed last month and a new one appears in the same spot, that is genuinely new — so it gets its own case.

5.3 Why every type has different numbers

This is the design decision people ask about most, and the answer is simple: different problems have different *shapes*.

Type	Same if within...	...and reported within	Reasoning
Pothole	25 m	60 days	A hole is one small spot, and stays for months
Streetlight	30 m	45 days	One pole, one lamp
Damaged property	30 m	60 days	A bench or railing — fixed spot, slow repair
Garbage overflow	50 m	3 days	Bins are emptied often, so a report two weeks later is a <i>new</i> overflow
Illegal dumping	60 m	7 days	Rubbish is spread over an area
Water leak	80 m	10 days	Water travels — you may see it far from the burst pipe
Broken drainage	80 m	30 days	Drains affect a whole stretch of road

The easy way to remember it. A pothole is a *dot* that lasts a long time: small distance, long window. A water leak is a *puddle* that gets fixed or dries up: big distance, short window.

5.4 How we search “nearby” quickly

The database stores latitude and longitude as plain numbers. It has no idea what a “metre” is. So we convert our metres into degrees before searching.

```
# Roughly 111,320 metres in one degree of latitude, everywhere on Earth.
lat_delta = radius_in_metres / 111320

# Longitude lines squeeze together as you move away from the equator,
# so the same number of degrees covers less ground. cos() corrects for that.
lon_delta = radius_in_metres / (111320 * cos(latitude))
```

Why the cos() is needed. Picture the lines running north–south on a globe. At the equator they are far apart. At the poles they all meet at a single point. So one degree of longitude is about 111 km at the equator but only about 96 km at the latitude of Delhi. Skipping this would make our search box too wide in northern cities.

Then we simply ask the database for anything inside that little rectangle:

```
WHERE category = 'POTHOLE'
AND latitude BETWEEN (my_lat - lat_delta) AND (my_lat + lat_delta)
AND longitude BETWEEN (my_lon - lon_delta) AND (my_lon + lon_delta)
AND created_at > (now - 60 days)
AND status NOT IN ('RESOLVED', 'REJECTED')
ORDER BY created_at ASC -- the oldest one wins
LIMIT 1
```

Sorting by oldest matters: new reports always join the **original** case, so a case never splits into a chain of copies.

5.5 An honest weakness — and why we accept it

We search a **square**, not a circle. The corners of a square stick out further than its sides, so a report about 35 m away can still match a 25 m rule if it sits diagonally.

Why we chose the square anyway. A true circle needs distance maths on every single row in the table. A square is two simple “between” comparisons, which the database can answer instantly using an index. And of the two possible mistakes, matching slightly too eagerly is far safer: a wrongly joined report is visible and an officer can split it, while a *missed* repeat silently recreates the original problem we set out to solve.

Index — like the index at the back of a textbook. Without one, finding a topic means reading every page. We built an index on (type, latitude, longitude) so this search stays fast even with millions of reports.

Part 6 — Sending it to the right desk, and deciding what matters

Features 6 and 7 — routing and priority

6.1 Choosing the department automatically

A pothole belongs to Roads. A burst pipe belongs to Water. The obvious way to code this is a chain of if-statements. We deliberately did not.

❌ Rules written in code

```
if category == "POTHOLE":
    dept = "ROADS"
elif category == "STREETLIGHT":
    dept = "ELECTRICAL"
elif ...
```

Every new type means editing code, testing it, and redeploying the whole website.

✅ Rules kept in a table

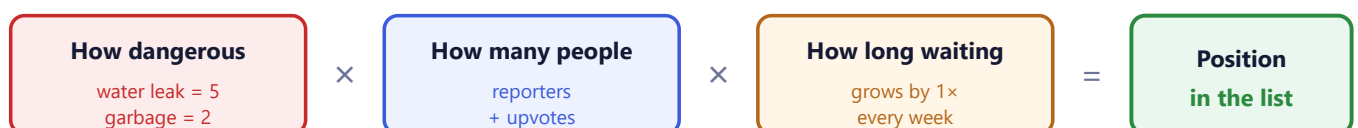
type	department
POTHOLE	Roads
WATER_LEAKAGE	Water

A new type is one new row in the database. No code change, no redeploy.

Six departments are set up automatically the first time the system runs. If a type somehow has no rule, it goes to **General Administration** so nothing is ever lost.

6.2 Deciding what to fix first

Oldest-first is unfair — a dangerous leak reported today matters more than a faded sign from last week. So each case gets a score, recalculated every time the officer opens the list.



```
score = danger × people × (1 + weeks_waiting)
```

Worked example — let us do the sum together

A water leak, reported 3 days ago, by 2 different people, with 1 upvote from a neighbour:

```
# danger for a water leak
danger = 5
# 2 reporters + 1 upvote
people = 2 + 1 = 3
# 3 days is 3/7 of a week
waiting = 1 + (3 / 7) = 1.43

score = 5 × 3 × 1.43 = 21.4 → shown as a red HIGH badge
```

Read what the formula is really saying. Multiplication means all three matter together. A very dangerous problem affecting one person can be outranked by a milder problem affecting twenty. And because the waiting part only ever grows, nothing can sit at the bottom of the list forever.

Part 7 — Deadlines that chase themselves

Feature 8 — SLA escalation ★

7.1 The problem with a to-do list

Even a perfectly sorted list has a bottom. Something can sit there for months while newer, louder problems keep arriving above it.

SLA — “Service Level Agreement”. A promise about *how fast* something will be handled. Ours is simply: this type of problem must be dealt with within this many days.

Type of problem	Must be handled within	Why
Overflowing garbage	2 days	Health hazard, and easy to clear
Water leak	3 days	Wastes water every hour it continues
Illegal dumping	4 days	Attracts more dumping if left
Broken drainage	5 days	Flood risk
Pothole / streetlight	7 days	Needs materials or a crew
Damaged public property	10 days	Real construction work

Notice something surprising. Garbage has the *lowest* danger score (2) but the *tightest* deadline (2 days). Danger and urgency are different questions. Danger asks “how bad is this?”; the deadline asks “how quickly should this reasonably be done?” Keeping them separate is what makes both behave sensibly.

7.2 What happens when a deadline passes

- 1 The case is marked as **late** (we save the exact moment it happened).
- 2 Its score is multiplied by **1.5**, pushing it up past comparable cases.
- 3 A red **⚠ SLA breached** badge appears on it in the officer's list.
- 4 A note is added to the case history, which the citizen can also see.
- 5 A message is sent to **General Administration** — a senior desk above the department that was supposed to handle it.

7.3 When does the check run? (a neat trick)

The obvious approach is a background program that wakes up every hour. We chose something simpler: **the check runs whenever an officer opens their dashboard, list, or charts.**

Why that is actually better here. A background program is one more thing that can quietly crash at 3 a.m. with nobody noticing — and on free hosting it may not be allowed to run at all. Tying the check to the moment an officer *looks* guarantees the list is correct exactly when someone is about to act on it. Nothing is ever seen in a stale state.

But won't it escalate the same case over and over?

No — and the reason is a single column. When a case escalates we write the timestamp into `escalated_at`. The search only ever picks up cases where that column is still empty.

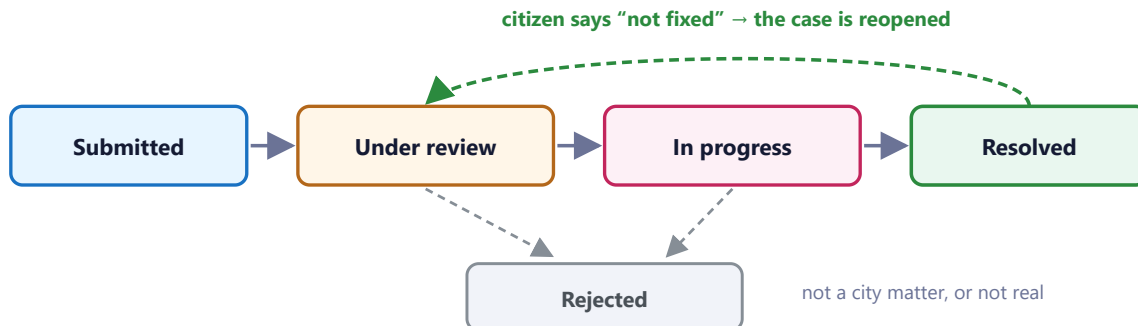
```
# "Find late cases that have NOT already been escalated"
WHERE status NOT IN ('RESOLVED', 'REJECTED')
  AND escalated_at IS NULL           ← the guard
  AND created_at < (now - deadline_for_this_type)
```

Idempotent — a fancy word worth knowing for your seminar. It means “running it again changes nothing”. Refreshing the page fifty times still escalates each case exactly once. Our one small column is what buys us that property.

Part 8 — Following a problem to the end

Features 9, 10 and 11 — status, history, proof, and the citizen's verdict

8.1 The five states



8.2 A diary that can never be edited

The current status is a single value — it gets overwritten each time it changes. So how do we know the story? Every change also **adds a new line to a diary**. Lines are only ever added, never changed or deleted.

Each diary line records: what it changed from, what it changed to, a note, an optional photo, **who did it** (citizen or city), and when.

Analogy. A bank statement. Your balance is one number that keeps changing, but the statement lists every transaction that got you there. Delete the statement and you can never explain the balance.

8.3 Proving the fix

When the work is done, the officer can add two things: a short note describing what was done, and a **photo of the repaired site**. Both appear on the citizen's page.

8.4 The citizen has the final word

This is our favourite feature, and it is a genuinely unusual one. When a case is marked resolved, the citizen is asked a simple question with two buttons:

✓ "Yes, it's fixed"

A line is added to the diary saying the reporter confirmed it. The case stays resolved. The loop is properly closed.

↩ "No, reopen it"

The case goes straight back to **Under review** and returns to the officer's list.

The important detail. Reopening acts on the whole *case*, not just one person's report. If five people reported that pothole and any one of them says it is not fixed, it reopens for everybody. A repair that did not work cannot be quietly closed by satisfying one person.

Part 9 — Seeing the whole city

Features 12 to 15 — upvoting, the map, the charts, the public report card

9.1 Upvoting someone else's report

On the map, any citizen can press 👍 on a problem someone else reported. Three rules:

- You cannot vote on your own report — your report already counts as one voice.
- Pressing again removes your vote. It is a toggle, not a counter.
- The database itself enforces one vote per person per case, so even a double-click cannot count twice.

Those votes flow straight into the `people` part of the priority formula from Part 6. Supporting a problem genuinely moves it up the queue.

9.2 The live map

One map component is written once and used by both citizens and officers — the officer version simply passes in a dark theme.

- Each case is a pin with the emoji for its type inside a coloured ring.
- A toggle switches the colouring between **status** and **type**.
- The map zooms itself to fit whatever pins exist.
- The legend is built from what is actually on screen — no empty entries.

A thoughtful detail worth mentioning in a seminar. We never rely on colour alone to show status. Resolved pins are also *faded and smaller*; rejected pins are *hollow with a dashed ring*. About one man in twelve has some colour blindness, and a shape difference works for everyone — including on a black-and-white printout.

9.3 Charts from real data

Every number on the officer's charts is counted from the database at the moment the page loads. Nothing is stored, cached or hard-coded.

Chart	What it answers
Donut — by status	How much is waiting versus done?
Bars — by type	What kind of problem does this city have most of?
Department table	Who has the biggest load, and who is fastest?
Hotspot list	Which streets keep generating problems?

One number is worth pointing at during a demo: **reports received** versus **problems (cases)**. The gap between them is exactly how much duplicate noise the system removed.

How hotspots are found

- 1 Round every location to 3 decimal places. That turns the city into a grid of squares roughly 110 m across.
- 2 Count the reports — not the cases — that fall into each square.
- 3 Throw away squares with fewer than 2 reports. One report is not a pattern.
- 4 Sort by count and show the worst 8.

Analogy. Rounding is like laying graph paper over a map. Everything inside one little square is treated as “the same place”, and then you just see which squares fill up.

9.4 The public report card ★

A web page anyone can open with no account, grading every department from A+ to D.

```
score = 50% × (how many were resolved)
       + 30% × (how many were resolved before the deadline)
       + 20% × (how fast on average)
```

Score	Grade
85 and above	A+
70 – 84	A
55 – 69	B
40 – 54	C
below 40	D

Two traps we deliberately closed

Trap 1 — the empty average

A department that resolves *nothing* has no resolution times to average. Treated carelessly, “no data” could look like “no delays”. So if nothing is resolved, the speed and on-time parts score **zero**, not blank.

Trap 2 — punishing the idle

A department with genuinely zero cases is not failing. It shows **“No data”**, never an F. Having nothing to do is not the same as doing nothing.

Why it cannot be gamed. The score reads the same rows the officers work in — there is no separate “public” copy to tidy up. And rejecting problems instead of fixing them lowers the resolution rate, so the shortcut makes the grade worse, not better.

Part 10 — The extra touches

Features 16 and 17 — the AI photo check and two languages

10.1 Letting an AI look at the photo

When a photo is uploaded we can send it to Claude, an AI that can see images, and ask a plain question:

For a new report — “**Does this picture plausibly show a pothole?**”
For a repair photo — “**Does this picture show the problem fixed?**”

The answer comes back as one of three verdicts, shown to the officer as a small badge:

Verdict	Meaning
 MATCH	The picture fits what was claimed
 MISMATCH	It might not — worth a human look
UNVERIFIED	We could not check (no key, or the service was unavailable)

Two design choices to highlight

- 1 **It runs afterwards, not during.** The citizen gets their tracking number immediately; the AI is asked in the background a moment later. Their submission never waits for it.
- 2 **It can never block a report.** No API key, service down, timeout, odd reply — every failure quietly becomes **UNVERIFIED**. The AI gives an officer a hint; it is never a gate, and it never has the final say.

Analogy. A spell-checker underlining a word. Helpful. But it does not stop you sending the email, and it is sometimes wrong about names.

10.2 English and हिन्दी

A  button switches the whole citizen side between English and Hindi — menus, the five form steps, type names, statuses, the timeline, the map legend. The choice is remembered on that device.

We wrote a tiny translation helper rather than installing a library. Every place text appears, the code calls:

```
t("nav.report", "Report")  
  ↑ the key      ↑ the English words, right there as a backup
```

Why keep the English text in the code? If a Hindi translation is ever missing, the screen shows the English word — never a blank space and never a raw code like `nav.report`. A half-finished translation still gives a completely usable app. Adding a third language means writing one new list of words and changing no screens.

Part 11 — Where everything is kept

The database, explained gently

11.1 What a table is

Analogy. A table is a spreadsheet with strict rules. Each **column** is a fixed question (“email address”, “date created”) and each **row** is one thing (one user, one report). Unlike a spreadsheet, the database refuses rows that break the rules — you cannot type “banana” into a date column.

We use seven tables. Here is what each one is for, in one line:

Table	Think of it as...
users	The register of everyone who can log in
issues	Every report ever submitted — the main table
status_history	The diary of everything that ever happened to a report
issue_votes	A tally sheet of who upvoted what
departments	The list of city departments and their email addresses
category_routes	The rule sheet: which type goes to which department
notifications	The outbox of messages to departments

11.2 How tables point at each other

Foreign key — a column holding the ID number of a row in another table. Instead of copying a person’s name onto every report, the report just stores `citizen_id = 42`. If person 42 changes their email, nothing else needs updating, because the truth lives in exactly one place.



11.3 The most important columns in `issues`

Column	What it holds and why it matters
<code>tracking_id</code>	The code shown to the citizen, e.g. <code>SMC-2026-000012</code>
<code>category</code>	The type. Decides distance, time window, danger score, deadline and department — five behaviours from one column
<code>latitude</code> / <code>longitude</code>	The pin. Used by the repeat-finder and the map
<code>status</code>	Where it is now, out of the five states
<code>case_id</code>	The group code. All reports of one problem share it — this is what makes duplicate grouping work
<code>is_primary</code>	True for the first report of a case. The officer's list only shows these, so one problem is one row
<code>escalated_at</code>	When the deadline was missed. Empty means “not yet late” — the guard that stops repeat escalation
<code>photo_verdict</code>	What the AI thought of the photo

11.4 Keeping the database tidy over time

Migration — a small numbered script describing one change to the tables, such as “add a column called `escalated_at`”. They run in order, so any teammate — or a fresh server — ends up with exactly the same structure by running one command.

We have seven, and reading their names is a short history of the project:

No.	What it added
001	Users and a basic issues table
002	Civic fields — type, location, photo, tracking number
003	Case grouping, so duplicates could be joined
004	Departments, routing rules and notifications
005	The history diary and repair evidence
006	Deadline tracking and upvotes
007	AI photo verdicts

Part 12 — One complete journey

Everything above, happening to a single pothole

Follow one problem from start to finish. Every step names the feature it demonstrates — this makes an excellent live demo script.

Step	What happens	Behind the scenes
1	Asha spots a pothole and opens the site on her phone	React loads; her token proves she is logged in
2	Picks “Pothole”, taps the map, adds a photo, types a sentence	Each step validated as she goes
3	Presses Submit	Photo checked by its fingerprint bytes; saved under a random name
4	—	Repeat check: nothing similar within 25 m in 60 days → a new case is created
5	—	Routing: pothole → Roads Department. A message is queued to them
6	Asha sees <code>SMC-2026-000012</code>	Her tracking number. Under a second has passed
7	—	<i>In the background:</i> the email goes out, and the AI checks the photo → MATCH
8	Ravi hits the same pothole next morning and reports it	Repeat found — joins Asha’s case. Now 2 people. Score rises
9	A neighbour presses 👍 on the map	3 people affected now
10	Officer Meena opens her list	Deadline check runs automatically. The case sits near the top: $4 \times 3 \times 1.1 \approx 13$
11	She sets it to “In progress”	A diary line is written: Submitted → In progress, by the city
12	Day 8 arrives with no repair	Deadline missed. Score $\times 1.5$, red badge, senior desk notified, diary line added
13	The crew fills the hole. Meena adds a note and a photo	AI checks the repair photo too. Status → Resolved
14	Asha sees “Is it fixed?” and taps Yes	Final diary line, written by the reporter. Loop closed
15	Anyone opens <code>/transparency</code>	Roads Department’s grade now includes this case — resolved, but late

Notice what nobody had to do. No human decided this was a duplicate. No human chose the department. No human noticed it was late. No human calculated a grade. Those four jobs — exactly the ones people are worst at doing consistently — happened by themselves.

Part 13 — Your seminar toolkit

How to present this, and what you will be asked

13.1 A 10-minute structure

Time	Say this	Show this
0–1 min	The pothole story from Part 1	Nothing — just talk. Get them nodding
1–2 min	The three pieces: dining room, kitchen, store room	The restaurant diagram
2–4 min	Submit a report live	The five steps, ending with the tracking number
4–6 min	Duplicate detection — your strongest card	Submit a second, nearby report. Show it joining the first
6–8 min	The officer's view: priority and the late badge	The work list and the  badge
8–9 min	Accountability	The public report card page
9–10 min	Close on the “nobody had to do this” point	The journey table from Part 12

13.2 Questions you will probably get

Question	A short, confident answer
"How do you know two reports are the same?"	Same type, close enough, recent enough, and the first one still open. The distance and time limits are different for each type of problem.
"Why a square search and not a circle?"	A square is two simple comparisons the database can index; a circle needs maths on every row. And matching a little too eagerly is the safer mistake.
"What if someone uploads a virus as a photo?"	We read the first few bytes of the file. Real images start with a known signature. Renaming a file does not change those bytes.
"Can a citizen open the admin pages?"	No. Every admin address checks the role on the server before doing anything. Hiding the link in the UI is not what stops them.
"Why no scheduled job for deadlines?"	The check runs whenever an officer looks at their list, so it is always correct when it matters — with nothing extra that can crash unnoticed.
"Doesn't the AI decide things?"	No. It writes an advisory badge for a human. If it is missing or wrong, everything still works exactly the same.
"Why not Google Maps?"	It needs an API key and billing. OpenStreetMap is free and open, and we can swap the map provider by changing one setting.
"Is it tested?"	169 automated tests — 110 on the backend, 59 on the frontend — covering permissions, duplicate detection, deadlines, scoring and the whole report flow.

13.3 The eight sentences worth memorising

- 1 One real problem = one case, no matter how many people report it.
- 2 Every type of problem gets its own distance, time window, danger score and deadline.
- 3 Order of work = danger × people affected × how long it has waited.
- 4 Miss the deadline and the case escalates itself to a senior desk — exactly once.
- 5 Status is one value; the history is a diary that can only be added to.
- 6 The citizen who reported it decides whether it is really fixed.
- 7 Nothing outside our control — geocoding, AI, email — is ever allowed to break a submission.
- 8 The public grade is calculated from the same rows the officers work in, so it cannot be tidied up.

If you remember only one thing. Every feature in this project exists to remove one specific human job that people do badly at scale: noticing repeats, choosing a department, judging urgency, chasing delays, and reporting honestly on yourself. The code is not clever for its own sake — each piece replaces a task that used to be forgotten.